

TASK プラズマ輸送ライブラリ化プロジェクト

日本語マニュアル・報告書

tr / ti / wr / wrx / fp モジュール (Phase L-0
～ L-7)

著者: k-yoshimi (東京大学)

日付: 2026 年 4 月 18 日

本書は TeX Live 不在の環境向けに matplotlib で組版した
搭乗便フォールバック版 PDF です. LaTeX ソースは
docs/manual/task-library-manual.tex を参照し
てください.

TeX Live があれば xelatex でより整った PDF が得られます.

[要旨]

本書は TASK コード群 (プラズマ統合輸送シミュレーション) のうち, 5 つの物理モジュール---{tr} (1 次元輸送), {ti} (不純物輸送), {wr} (波動レイトレーシング), {wrx} (拡張レイトレーシング), {fp} (Fokker--Planck 解析)---を {共有ライブラリ化} し, {Python から利用可能} にした一連の作業 (Phase L-0 ~ L-7) を, Fortran や ctypes に馴染みのない初学者にも読めるようにまとめたマニュアルである.

使い方章 (第 3~7 章) は{です・ます調}で平易に記述し, 技術章 (第 8~10 章) は{だ・である調}で手短にまとめる.

[要旨終わり]

第 はじめに 章

■ [学習目標]

本章では、本書が何であるかと、読者がどんな力を得られるかを整理します。ここは読み飛ばさずに目を通しておくと、後の章で迷子になりにくくなります。

本書の目的

TASK コード群 (京都大学・東京大学で長年開発されているプラズマ統合シミュレーションコード) の一部モジュールが、2026 年 4 月の集中開発で「共有ライブラリ (`.so` ファイル)」として配布・利用できるようになりました。これによって、

- Fortran の `tr2` や `fp` といった対話型プログラムを起動しなくても、
- Python スクリプトから {関数呼び出し感覚} でプラズマ輸送計算を回せる、

ようになりました。

本書はその {使い方} と {作業内容} を記録するマニュアル兼報告書です。

読者対象 (いちばん大事なところ)

次のような方を想定しています。

- プラズマ物理に興味がある学生、ただし Fortran は書いたことがない
- Python は少し触ったが、`ctypes` や共有ライブラリは聞いたことがない
- ターミナルで `cd`, `ls`, `make` くらいは使える

「C, Fortran, 共有ライブラリ」などの用語は {最初に出てきた場所で毎回説明します}。分からない用語が出てきたら、近くの [補足] や [FAQ] を確認してください。

本書で何ができるようになるか

読み終えたときに, 次のことが自分の手で出来るようになります.

- TASK のソースから `libtrapi.so` などの共有ライブラリをビルドする
- Python スクリプト 5 行で `tr / ti / wr / wrx / fp` のいずれかの計算を回し, 結果を dict として受け取る
- パラメータ名と値を指定してパラメータスキャンループを書く
- 4 層の回帰テスト (`run\_tests.sh`) で自分の変更が Phase 0 ベースラインと $1e-10$ で一致することを確認する

前提ソフトウェア

- GNU/Linux (Ubuntu 22.04 以降で検証) もしくは同等 Unix
- gfortran 9 以降, gcc 9 以降, GNU Make
- Python 3.8 以降 (標準ライブラリのみで動作. NumPy は任意)
- Git

詳細な OS パッケージは `INSTALL\_Ubuntu.md` を参照してください.

本書の構成

- 第 (参照) 章 — 全モジュール共通の構造. TikZ 3 層図と 5 関数 API の説明.
- 第 (参照) ~ (参照) 章 — `tr / ti / wr / wrx / fp` の各モジュール. 初学者向け hello-world と FAQ を含む.
- 第 (参照) 章 — 4 層テスト基盤.
- 第 (参照) 章 — 作業状況 (完了/未完了, eq/tot 除外理由).
- 第 (参照) 章 — 付録 (Makefile ターゲット, PR 一覧, 参考資料).

表記規約

- ``\$` はシェルのプロンプトです. ``\$` の後ろをターミナルに打ち込んでください.
- ``>>>` は Python の対話プロンプトです.
- コード中の日本語コメントは, あえて全角で書くことで実際の挙動と区別しています. 実コードには入れないでください.

第 共通アーキテクチャ 章

■ [学習目標]

本章では, `tr / ti / wr / wrx / fp` のすべてに共通する
3 層構造,
5 関数 C ABI, エラーコード 0~4, PIC ビルド, Python
`ctypes` の
2 層設計を学びます. 以降の章はすべてこの章の応用です.

全体像 — 3 層アーキテクチャ

すべての ``X`lib` (`X` は `tr / ti / wr / wrx / fp`
のどれか) は
同じ 3 層構造を持ちます.

lem

[図 (TikZ): PDF 版では LaTeX 層で描画]

各層の役割 (やさしい説明)

[ユーザ層] あなたが書くコード. Python の関数呼び出しと
同じ感覚で書けます.

[Python ラッパ層] Python から C の関数を呼び出すための
「翻訳係」. 内部で ``ctypes`` を使って ``so``
ファイルを読み込みます.

[C ABI 層] C 言語の関数呼び出し規約 (Application Bin
ary

Interface) で Fortran と Python/C
を{つなぐ窓口}.

わずか 5 個の関数に絞ってあるので覚えやすいです.

[Fortran バックエンド] 物理計算の本体. 既存の ``tr2``
バイナリと{全く同じソースコード}を使うので, 数値が
一致することが保証されます.

[共有ライブラリ] 上記全部をひとまとめにしたバイナリ
ファイル. 拡張子 ``so`` は ``shared objec
t`` の略.

共有ライブラリ (.so) とは

■ [補足]

{共有ライブラリ} (`.so` ファイル) とは, 「プログラムが実行中に読み込んで使う, 再利用可能なコードのかたまり」 のことです. Windows の `dll`, macOS の `dylib` に相当します.

通常のプログラムは自己完結した `.exe` で全機能を持ちますが, 共有ライブラリ方式では

- Python から `ctypes.CDLL("libtrapi.so")` と書けば読み込める
- C から `dlopen()`, コンパイル時リンクは `gcc -ltrapi` で OK

という具合に{あとから好きな言語から呼べる}のが利点です.

5 関数 C ABI

各モジュール `X` につき, エクスポートされる C 関数はたった 5 つです. これを覚えれば `X` の頭文字さえ差し替えれば 5 モジュールすべてに共通です.

```
int tr_init(void);           /* 初期化
*/
int tr_run(int ntmax);       /* 時間発展
*/
int tr_set_param(const char* name, double value); /* パラメタ
*/
int tr_get_state(tr_state_t* state); /* 状態取得
*/
int tr_finalize(void);       /* 後始末
*/
```

いずれも返り値 `int` は{エラーコード} (次節参照) です.

エラーコード 0~4

C ABI の返り値は次の 5 段階しかありません.

覚えやすいので暗記してしまってもかまいません.

[表]

| rll@{}

{コード} | {定数名} | {意味}

0 | `X\_OK` | 正常終了

1 | `X\_ERR\_INVALID` | パラメータ名/値が不正

2 | `X\_ERR\_NOT\_INIT` | `X\_init` を呼んでいない

3 | `X\_ERR\_CALC\_FAILED` | 計算が失敗した

4 | `X\_ERR\_NOT\_IMPL` | 未実装 (Phase L-2 時点のスタブ) |

[/表]

Python 側では, 0 以外が返ると `X lib` `Error` の {派生例外} が投げられます. 例: `TrlibParamError`, `FplibNotInitError`.

5 関数のライフサイクル

[図 (TikZ): PDF 版では LaTeX 層で描画]

- `X\_init` — 1 プロセス 1 回だけ呼ぶ. Fortran 側の COMMON ブロックを初期化します.
- `X\_set\_param` — 何度でも呼べます. 例: `"RR"` (大半径), `"BB"` (磁場), `"PN[1]"` (1 番目のイオン密度) に値を与えます.
- `X\_run` — 指定したステップ数だけ時間発展を進めます.
- `X\_get\_state` — 現在の状態を `X\_state\_t` 構造体 (C) に書き出します. Python では `XState` dataclass になります.
- `X\_finalize` — 片付け. 通常は `with` 文が自動で呼んでくれます.

PIC (位置独立コード) とは

■ [補足]

{PIC} = Position Independent Code. 共有ライブラリ (`.so`) を作る時に{どのアドレスにロードされても動く}のように gcc/gfortran が吐くコードのことです. 通常の静的ライブラリ (`.a`) には要りません.

TASK では, 旧来の `.a` アーカイブ (非 PIC) を保存したまま, PIC 版 `lib\*\_pic.a` を並行して生成するように各モジュールの `Makefile` を修正しました. これで `tr2` バイナリのビルドは一切変わらず, 共有ライブラリも作れるようになっています.

PIC 用の Makefile ターゲット

各モジュールには次のターゲットが追加されています.

```
make -C lib libs_pic # lib/ 配下の PIC アーカイブ
make -C pl libs_pic # pl/plcomm_pic.a など
make -C eq libs_pic
make -C mt xp libs_pic
make -C bpsd libs_pic
make -C tr libtrapi.so # 最終成果物
```

Python ラッパの 2 層設計

Python 側は{わざと 2 層}に分けてあります.

[図 (TikZ): PDF 版では LaTeX 層で描画]

低レベル `\_ffi.py` は C ABI の{鏡写し}で, `ctypes.Structure`

で `X\_state\_t` の各フィールドを同じ順序・同じ型で並べています. バイト単位で一致するので, Fortran が書き出した生データをそのまま Python 側で読めます.

高レベル `Xlib.py` は Python らしい API (`with` 文, `dataclass`, 例外クラス) を提供します.

ライブラリの探し方

`Xlib()` を引数なしで呼ぶと, 次の順で `.so` を探します.

- 環境変数 `XLIB\_PATH` (例: `TRLIB\_PATH`, `FP
LIB_PATH`)
- `<リポジトリ root>/X/libXapi.so` (標準ビルド場所)
- `<リポジトリ root>/lib/libXapi.so` (将来のインスト
ール先)

明示的に指定したい場合は `Trlib(lib\_path="/絶対/パス")`
とします。

RTLD_LAZY ロード

■ [補足]

`ctypes.CDLL(..., mode=RTLD\_LAZY)` は「実際に
その関数が呼ばれる
瞬間までシンボル解決を遅らせる」オプションです。TASK の共有ライブラリ
は、グラフィクス関連のシンボルが一部残ったまま作られるため、
`RTLD\_NOW` (即時解決) だとロードに失敗することがあります。
`RTLD\_LAZY` なら、呼ばないパスの解決を後回しにできるので安全です。

メモリレイアウト (C と Fortran の違い)

■ [注意]

C は{行優先}, Fortran は{列優先}でデータを並べます。
たとえば C の `double RN[NRMAX][NSMAX]` は For
tran の
`REAL(KIND=8) :: RN(NSMAX, NRMAX)` と{バイト
単位で一致}
しますが、添字の順序が逆です。Python 側の `XState` data
class は
`nrmax` 側を外側のリスト, `nsmax` を内側のリストとして
見せるので、C 流の順番です。

本章のまとめ

これで第 (参照) 章以降は、「API は 5 つ, エラーは 0~4, 例外
は

`X lib` `Error` 派生, `with` で書く」という
{1 つのパターンを 5 回繰り返す}だけで読み進められます.

第 tr モジュール 章

■ [学習目標]

本章では, TASK/TR (1 次元トカマク輸送シミュレーション) を Python から呼び出すライブラリ `trlib` の使い方を学びます. 共有ライブラリのビルド, 最短 hello-world (5 行), パラメータの与え方, FAQ, そしてテストを実行する方法までカバーします.

概要 — tr は何をする

{TASK/TR} は, トカマクや球状トカマクの {1 次元 (半径方向) 輸送シミュレーション} を行うモジュールです. 「プラズマの半径方向の分布が, 時間とともにどう変化するか」を, 粒子数・温度・電流・磁場の平衡を解きながら追跡します.

得られる代表的な量は

- `RN[i][j]` — 半径点 (i) での (j) 番目の粒子種の密度
- `RT[i][j]` — 同じく温度
- `AJ[i]` — 電流密度プロファイル
- `QP[i]` — 安全係数 (q) プロファイル
- スカラー: `T` (時間), `WPT` (蓄積エネルギー),
`Q0` (中心 q), `BETAN` (β_N) など
13 種

です.

前提とビルド

ライブラリをビルドする

```
$ cd /home/you/task          # TASK リポジトリのルートに移動
$ make -C lib libs_pic       # 下位ライブラリの PIC 版を作る
$ make -C pl libs_pic        # プラズマ共通モジュール
$ make -C eq libs_pic        # 平衡モジュール
```

```
$ make -C mt xp libs_pic      # 疎行列ソルバ
$ make -C bpsd libs_pic      # BPSD データ橋渡し
$ make -C tr libtrapi.so      # tr モジュールの共有ライブラリ本体
```

最後に `tr/libtrapi.so` というファイルが出来ていれば成功です。

```
$ ls -lh tr/libtrapi.so
-rwxr-xr-x 1 user user 8.2M 4月 18 23:00 tr/libtrapi.so
```

ファイルサイズは環境によって異なりますが、だいたい 5 ~ 10 MB くらいです。

エクスポートされている関数を確認する

「本当に 5 つの関数が入っているか」は `nm` コマンドで確認できます。

```
$ nm -D tr/libtrapi.so | grep 'T tr_'
0000000000005a1b0 T tr_finalize
0000000000005a090 T tr_get_state
00000000000059e10 T tr_init
00000000000059f80 T tr_run
00000000000059d30 T tr_set_param
00000000000059ca0 T tr_set_param_str
```

`T` 列のシンボルが「エクスポートされた関数」を示します。

Python から見えるようにする

```
$ export PYTHONPATH=/home/you/task/python:$PYTHONPATH
```

こうすると `import trlib` が通ります。

最短 hello-world (5 行)

```
from trlib import Trlib      # (1) インポート
with Trlib() as tr:          # (2) 初期化 (= tr_init 自動)
    tr.set_params(RR=3.0, BB=3.0) # (3) 大半径 3m, 磁場 3T
    tr.run(ntmax=10)           # (4) 10 ステップ時間発展
    state = tr.get_state()      # (5) 状態を取得 (型は TrState)
    print(state.scalars["T"])   # 最終時刻 (秒)
```

行ごとの解説

[(1) `from trlib import Trlib`] パッケージ `trlib` からクラス `Trlib` を読み込みます。

`Trlib` は共有ライブラリのハンドル (持ち手) です。
 [(2) `with Trlib() as tr:`] `with` 文は Python のコンテキストマネージャ機能です。入る時に
 `\_\_enter\_\_` (= `tr\_init`), ブロックを抜ける
 きに
 `\_\_exit\_\_` (= `tr\_finalize`) が自動で呼ばれます。
 後片付けを忘れる心配がありません。
 [(3) `tr.set\_params(RR=3.0, BB=3.0)`] `set\_params` はスカラー値を一括でセットするヘルパーです。
 Fortran の `/TR/` ネームリストと同じ名前
 (`RR` = 大半径, `BB` = トロイダル磁場) をキーワード
 引数で指定します。
 [(4) `tr.run(ntmax=10)`] 時間発展を 10 ステップ進めます。ステップ幅は `DT`
 (デフォルト 0.01 秒) です。
 [(5) `state = tr.get\_state()`] 現在の状態を `TrState` データクラスに写し取ります。
 `state.scalars["T"]` で現在時刻, `state.R`
 `T[i][j]`
 で温度プロファイルが取れます。

期待される出力例

```
$ PYTHONPATH=python python3 examples/quickstart.py
NT=50 NRMAX=50 NSMAX=2
T = 0.5
WPT = 8.3e+05
Q0 = 0.96
BETAA= 0.42
```

パラメータの指定方法

3通りの指定方法があります。

方法 A – スカラー (`set\_params`)

Python のキーワード引数で一括指定できます。

```
tr.set_params(RR=7.5, RA=2.0, BB=5.3, DT=0.05, NTMAX=200)
```

方法 B — 配列要素 (`set_param`)

キーワード引数には `` `` が使えないので、配列要素は `set_param` を使います. 1-origin (1 始まり) です.

```
tr.set_param("PN[1]", 1.0) # 1 番目の粒子種の密度
tr.set_param("PN[2]", 1.0) # 2 番目
tr.set_param("CDW[12]", 0.5) # 配列 CDW の 12 番目
```

方法 C — 文字列パラメータ (`set_param_str`)

TR には平衡データファイル名 `KNAMEQ` など文字列パラメータがあります. これは専用 API で与えます.

```
tr.set_param_str("KNAMEQ", "eqdata.ITER01")
```

コンテキストマネージャ `with`

の解説

■ [補足]

Python の `with` 文は「ブロックに入るときと抜けるときに必ず後片付けを走らせる」仕組みです. ファイル操作の `with open(...) as f:` と同じ発想です.

`Trlib` も `__enter__` / `__exit__` を実装してあるので、例外が発生しても `tr_finalize` が必ず呼ばれ、メモリやグローバル状態が掃除されます.

明示的に書くなら次と同じ意味です:

```
tr = Trlib()
try:
    tr.set_params(RR=3.0)
    tr.run(ntmax=10)
    state = tr.get_state()
finally:
    tr.close()
```

FAQ / つまづきどころ

■ [FAQ] Q1. `FileNotFoundError: libtrapi.so not found ...` が出る

まだビルドしていないか, 場所が違います. 次を試してください:

- `ls tr/libtrapi.so` が存在するか確認
- 無ければ `make -C tr libtrapi.so` を実行
- どうしても別の場所に置きたければ
 `export TRLIB\_PATH=/path/to/libtrapi.so`

■ [FAQ] Q2. `TrlibParamError: ierr=1` が出る

存在しないパラメータ名を指定した, または配列の添字が範囲外です.
`tr/tr\_param\_registry.f90` の `SELECT CASE` 節に
登録されているかを確認してください. 追加するには Fortran 側に
1 行足すだけです (再ビルドが必要).

■ [FAQ] Q3. `set\_params(PN\_\_1=1.0)` と書いてしまった

`\_\_` (アンダースコア 2 つ) を含むキーは, 配列構文の書き間違い
とみなして明示的にエラーを出します. 正しくは
`tr.set\_param("PN[1]", 1.0)` です.

■ [FAQ] Q4. 同じプロセスで 2 個の `Trlib()` を作れる?

作れますが{意味がありません}. Fortran 側は COMMON ブロックで
グローバル状態を持つので, 2 個目の `tr\_init` は最初の状態を
上書きします. プロセス分離 (`multiprocessing`) を使ってください.

■ [FAQ] Q5. 結果が `tr2` (CLI 版) と一致しない

回帰テスト `trlib\_equivalence` を実行して Phase 0
ベースライン
との差分を確認してください.


```
bash test_run/run_tests.sh trlib_equivalence
```

tol は `1e-10`. 許容値以上ずれる場合は, おそらく登録漏れのパラメータを指定していません.

■ [FAQ] Q6. NumPy が必要?

必要{ありません}. `TrState` は Python の `list` です. NumPy を使いたい場合は
`import numpy as np; np.array(state.RT)`
で変換できます.

サポートされているパラメータ

`tr/tr\_param\_registry.f90` に登録されている主なパラメータを表 (参照) に示します. 名前は Fortran の `/TR/` ネームリストと一致しています.

[表]

| p{3.3cm}p{2.2cm}p{9cm}@{}
 キャプション: tr モジュールの主な登録パラメータ

{名前} | {型} | {意味}

{名前} | {型} | {意味}

`RR` | double | プラズマ大半径 [m]

`RA` | double | 小半径 [m]

`RKAP` | double | elongation (縦横比)

`RDLT` | double | triangularity

`BB` | double | トロイダル磁場 [T]

`PHIA` | double | 全磁束 [Wb]

`RIPS` | double | 開始時刻のプラズマ電流 [MA]

`RIPE` | double | 終了時刻のプラズマ電流 [MA]

`MODELG` | int | 幾何モデル (1:解析, 3:eqdata)
`NSMAX` | int | 粒子種の数
`PA[i]` | double 配列 | 質量数 (i 番目の粒子種)
`PZ[i]` | double 配列 | 電荷
`PN[i]` | double 配列 | 中心密度
`PNS[i]` | double 配列 | 縁密度
`PT[i]` | double 配列 | 中心温度
`PTS[i]` | double 配列 | 縁温度
`DT` | double | 時間ステップ [s]
`NTMAX` | int | 時間ステップ数
`NTSTEP` | int | 出力間引き
`EPSLTR` | double | 反復収束判定
`LMAXTR` | int | 最大反復数
`MDLKAI` | int | 輸送モデル (カイ) 選択
`MDLETA` | int | 抵抗モデル
`MDLAD` | int | 輸送項切替
`CDW[i]` | double 配列 | 輸送係数 (12 要素)
`CHP`, `CK0`, `CK1` | double | カイ補正係数
`MDLNB`, `MDLEC`, `MDLLH`, `MDLIC` | int
| NBI/EC/LH/IC 加熱モデル
`MDLJBS` | int | ブートストラップ電流モデル
`MDLPEL`, `MDLST`, `MDLNF`, `MDLUF` | in
t | ペレット/ソース/核融合/UFILE
`PROFN1`, `PROFN2` | double | 密度プロファイル形状
`PNC`, `MDLIMP` | -- | 不純物混入
`PNBR0`, `PNBRW`, `PNBENG`, `PNBRTG` | d
ouble | NBI 位置/幅/エネルギー

```
`PICCD`, `PICR0`, `PICRW`, `PICNPR` | do
uble | ICRF パラメータ

`PECCD`, `PECR0`, `PECRW`, `PECNPR` | do
uble | ECRF

`PLHCD`, `PLHR0`, `PLHRW`, `PLHNPR`, `PL
HTOT` | double | LH

`NGTSTP`, `NGRSTP` | int | グラフ出力間引き

`KNAMEQ` | 文字列 | 平衡データファイル名 (`set_param_
str`) |
[/表]
```

`TrState

出力`

`tr.get\_state()` は `TrState` dataclass を返します。
フィールドは `tr/tr\_api.h` の C 構造体 `tr\_state\_t` と対応します。

```
state.nt          # 時間ステップ数
state.nrmax       # 実働の半径点数 (<= 500)
state.nsmax       # 実働の粒子種数 (<= 8)
state.scalars["T"] # 時間 (秒)
state.scalars["WPT"] # 蓄積エネルギー
state.scalars["Q0"] # 中心 q
state.scalars["BETAN"] # beta_N
state.RN[0][0]    # RN[半径=0][粒子=0]
state.RT          # [nrmax][nsmax] の 2 次元リスト
state.AJ[0]       # 電流密度 (先頭)
state.QP[-1]      # 端 q
state.to_dict()   # JSON 化用 dict
```

変更履歴 — Phase L-0 ~ L-7

[表]
| lllp{6.5cm}@{}}

Phase | PR | 日付 | 内容

L-0 | #2 | 2026-04-18 | Phase 0 回帰テスト基盤:
`tr\_iter01`, `tr\_m0904`, `tr\_tst2` 3 ケー
スの JSON ベースライン

L-1 | #21 | 2026-04-18 | `tr/Makefile` の
グラフィクス分離. `tr2` はそのまま, `libtrapi.so` はグラ
フィクスフリー

L-2 | #27 | 2026-04-18 | C ABI 基盤 (`tr_a
pi.h`, 5 関数スタブ ierr=4)

L-3 | #33 | 2026-04-18 | パラメータレジストリ (`tr
_param_registry.f90` 38 ケース) および `tr_ini
t/run/get_state/finalize` 実装

L-4 | #35 | 2026-04-18 | `make -C tr lib
trapi.so` で `.so` が生成. `lib*_pic.a` を各所に
追加

L-5 | #44 | 2026-04-18 | Python ラッパ `pyt
hon/trlib/`: `Trlib`, `TrState`, 例外階層, `
_ffi`

L-6 | #53 | 2026-04-18 | 4 層テスト: `trlib_
equivalence`, `_c_abi`, `_ffi`, `_wrappe
r`, `_sweep`

L-7 | #--- | 2026-04-18 | ドキュメント: `pytho
n/trlib/README.md`, `examples/*.py`, arc
hitecture.md |
[/表]

テスト

4 層の回帰テストを用意しています (第 (参照) 章参照).

```
# 単一層
bash test_run/run_tests.sh trlib_equivalence # Layer 1
bash test_run/run_tests.sh trlib_c_abi      # Layer 2
bash test_run/run_tests.sh trlib_ffi        # Layer 3 (低レベ  
ル)
bash test_run/run_tests.sh trlib_wrapper    # Layer 3 (高レベ  
ル)
bash test_run/run_tests.sh trlib_sweep      # Layer 4
```

Phase L-5/L-6 での既知の問題: `tr\_m0904` ベースライン
は
 $\backslash(10^{\{-8\}}\backslash)$ の僅かなドリフトがあり, ベースライン側で pin し
て
います. `tr\_iter01` と `tr\_tst2` は $1e-10$ で一致
します.

第 ti モジュール 章

■ [学習目標]

本章では, TASK/TI ({不純物輸送}モジュール, 多価電荷イオンの輸送と原子過程をあわせて解く) を Python から呼ぶ `tilib` の使い方を学びます. TR と非常に似た API 体系です.

概要 — ti は何をする

{TASK/TI} は, TR が扱わない{不純物 (高 Z イオン)}の輸送と, 各電荷状態間の電離・再結合を解きます. たとえばアルゴン ($Z=18$) やタングステン ($Z=74$) が壁から入ってきてどう分布するかをシミュレーションします.

前提とビルド

TR と同じ手順です.

```
$ cd /home/you/task
$ make -C lib libs_pic
$ make -C pl libs_pic
$ make -C eq libs_pic
$ make -C adpost libs_pic      # 原子過程データ
$ make -C open-adas/adf11/adf11-lib libs_pic
$ make -C mt xp libs_pic
$ make -C bpsd libs_pic
$ make -C ti libtiapi.so
```

■ [注意]

`libtiapi.so` のトップレベル `make` ターゲットは執筆時点で develop にマージされておらず, フィーチャブランチ `feature/ti-library-L4-shared-lib` に存在します. マージ待ちの状態でも Python ラップ側はライブラリが無ければ FFI テストをスキップするので, 他の章の学習は進められます.

最短 hello-world

```
from tilib import TiLib          # クラス名は TiLib (大文字
L)
with TiLib() as ti:
    ti.set_params(RR=6.2, BB=5.3, NSMAX=2)
    ti.set_param("PN[1]", 0.7)
    ti.run(ntmax=10)
    state = ti.get_state()
print("T =", state.T)           # ti は T を dataclas
s 属性で持つ
print("RTA[0] =", state.RTA[0]) # 温度プロファイル
```

TR との違い

- クラス名が `TiLib` (大文字 L). エイリアス `Tilib` もエクスポート済み.
- スカラー `T` は `state.scalars` でなく `state.T` のように直接属性アクセスできます.
- 収束診断用に `residual\_loop\_max`, `icount\_loop\_max`, `icount\_mat\_max` が `state` 直下にあります.
- 密度は `state.RNA`, 温度は `state.RTA`, 速度は `state.RUA` (それぞれ `[nrmax][nsa\_max]`).

パラメータ指定

TR と同じ要領です. 2 次元配列は `"NAME[i,j]"` でも指定可能です.

```
ti.set_param("PA[3]", 40.0)      # 3 番目の粒子種の質量数
ti.set_param("MODEL_BND[1,3]", 1.0) # 2 次元配列
ti.set_param("BND_VALUE[1,3]", 1.0e-2)
```

■ [補足]

ti モジュールの `/TI/` ネームリストは, 歴史的な経緯で `PM` (質量) のキーが `plcomm` の `PA` に `pm=>pa` でリネームされていました. Phase L-3 以降は `PA` を {正式な API 名} として採用し, `PM` は

受け付けません. Argon の fixture は ``"PA[3]" = 40`
`.0,`
`"NPA[3]" = 18`` で指定してください.

`TiState

の主なフィールド`

```
state.nt, state.nrmax, state.nsa_max, state.nsmx
state.T          # 時間
state.residual_loop_max # 収束診断
state.icount_loop_max, state.icount_mat_max
state.RNA        # [nrmax][nsa_max] 密度
state.RTA        # [nrmax][nsa_max] 温度
state.RUA        # [nrmax][nsa_max] 速度
state.RBP, state.RQP # [nrmax] 磁場, 安全係数
state.RJP, state.ZEFF # [nrmax] 電流, Z_eff
state.BETA, state.BETAP # [nrmax] beta, beta_p
```

FAQ / つまづきどころ

■ [FAQ] Q. `PM[3]

```を指定したがエラー]  
 上記 note 参照. ``PA[3]`` を使ってください.

### ■ [FAQ] Q. `DN0`, `DT0` がレジストリにない

Phase L-3 時点では登録されていません. Fortran 側  
``ti_param_registry.f90`` に ``CASE("DN0")``  
 を 1 行追加すれば  
 拡張できます (再ビルド要).

### ■ [FAQ] Q. `ti\_ar` のベースライン比較が微妙にずれる

``ti_ar`` は ``tr_m0904`` と同様の軽微ドリフトがあり,  
 ベースライン側で pin しています. ベースラインを再生成する場合は,  
``tools/extract_ti_metrics.py`` でダンプ後に dif  
 f してください.



## 入力パラメータ表 (抜粋)

[表]

| p{3cm}p{2.2cm}p{9.2cm}@{}  
キャプション: ti モジュールの主な登録パラメータ

{名前} | {型} | {意味}

`RR, RA, RKAP, RDLT, BB, RIP` | double |  
装置幾何

`NSMAX` | int | 粒子種の数

`PA[i]`, `PZ[i]`, `PN[i]`, `PNS[i]`, `PT  
[i]`, `PTPR[i]`, `PTPP[i]`, `PTS[i]`, `P  
U[i]`, `PUS[i]` | double 1D | 粒子種ごとの物理量

`NPA[i]`, `ID\_NS[i]` | int 1D | 原子番号, ID

`PROFN1[i]`, `PROFN2[i]`, `PROFT1[i]`, `  
PROFT2[i]`, `PROFU1[i]`, `PROFU2[i]` | d  
ouble 1D | プロファイル形状

`DT`, `NRMAX`, `NTMAX`, `NTSTEP`, `NGTST  
EP`, `NGRSTEP` | 数値 | 時間・メッシュ

`MAXLOOP`, `EPSLOOP`, `EPSMAT`, `MATTYPE  
` | 数値 | 非線形反復

`MODEL\_EQB`, `MODEL\_EQN`, `MODEL\_EQT`, `  
MODEL\_EQU` | int | 平衡・物理量モデル

`MODEL\_KAI`, `MODEL\_DRR`, `MODEL\_VR` | i  
nt | 輸送モデル

`MODEL\_NC`, `MODEL\_NF`, `MODEL\_NB`, `MOD  
EL\_EC`, `MODEL\_LH`, `MODEL\_IC` | int | 各  
加熱機構

`MODEL\_CD`, `MODEL\_SYNC`, `MODEL\_PEL`, `  
MODEL\_PSC` | int | 電流駆動/シンクロトロン/ペレット/源

`NZMIN\_NS[i]`, `NZMAX\_NS[i]`, `NZINI\_NS[  
i]` | int 1D | 電荷状態の最小/最大/初期

`MODEL\_BND[i,j]`, `BND\_VALUE[i,j]` | 2D

| 境界条件

`PROFJ1`, `PROFJ2` | double | 電流プロファイル形状

|

[/表]

## 変更履歴 — Phase L-0 ~ L-7

[表]

| lllp{6.5cm}@{}}

Phase | PR | 日付 | 内容

L-0 | #12 | 2026-04-18 | ti 用回帰テスト基盤 (`ti\_min`, `ti\_ar`, `ti\_w`)

L-1 | #22 | 2026-04-18 | `ti/Makefile` C  
ORE / GRAPHICS / MENU 分割

L-2 | #28 | 2026-04-18 | `ti\_api.h` + スタ  
ブ, `tests/c\_abi/`

L-3 | #34 | 2026-04-18 | パラメータレジストリ (35  
ケース, 1D/2D) と実装

L-4 | #41 | 2026-04-18 | `libtiapi.so` (  
フィーチャブランチ staging)

L-5 | #47 | 2026-04-18 | Python ラップ `pyt  
hon/tilib/`

L-6 | #57 | 2026-04-18 | 4 層テスト `tilib\_\*`  
、

L-7 | #64 | 2026-04-18 | ドキュメント整備 |

[/表]

## テスト実行

```
bash test_run/run_tests.sh tilib_equivalence tilib_wrapper \
 tilib_ffi tilib_c_abi tilib_sweep
```

## 第 wr モジュール 章

### ■ [学習目標]

本章では, TASK/WR ({波動レイトレーシング}: EC/LH 波の軌道追跡とパワー吸収) を Python から呼ぶ `wrlib` の使い方を学びます. 出力は「レイごとのピーク位置」や「半径プロファイル」といった独特の形になります.

### 概要 — wr は何をする

{TASK/WR} は, 電子サイクロトロン (EC) 波や低域混成 (LH) 波などの高周波がプラズマ中でどう伝わり, どこで吸収されるかを幾何光学的レイトレーシングで追跡します. 結果としてレイごとの終端 (`rays\_end`) や, 半径方向のパワー堆積プロファイル (`pos\_nrs`, `pwr\_nrs`) が得られます.

### 前提とビルド

```
$ cd /home/you/task
$ make -C lib libs_pic
$ make -C pl libs_pic
$ make -C eq libs_pic
$ make -C dp libs_pic # 分散関係モジュール (必要)
$ make -C mtxp libs_pic
$ make -C bpsd libs_pic
$ make -C wr libwrapi.so
```

### 最短 hello-world

```
from wrlib import Wrlib
with Wrlib() as wr:
 wr.set_params(RR=3.0, BB=3.5, NSMAX=2)
 wr.set_param("PN[1]", 0.7)
 wr.run(nray_request=0) # 0: ネームリストの NRAYMAX
 を使う
 state = wr.get_state()
print("pwrmax_rs =", state.scalars["pwrmax_rs"])
print("rays_end[0] =", state.rays_end[0])
```

### ■ [補足]

`nray\_request` は `wr\_run` の引数で、レイ本数の上書き用です。0 を渡すとネームリスト (`NRAYMAX`) の値をそのまま使います。

## `WrState`

の構造`

3 つの実行時次元を持ちます。

- `nraymax` — レイの数
- `nrsmx` — 半径 (rs) プロファイルの点数
- `nrlmx` — 半径 (rl) プロファイルの点数

```
state.scalars["pos_pwrmax_rs"] # rs 軸での全体ピーク位置
state.scalars["pwrmax_rs"] # そのパワー
state.nstp_end[i] # レイ i の終了ステップ
state.rays_end[i] # [9] 要素 (RAYS(0..8, i))
state.pos_nrs[k], state.pwr_nrs[k]
state.pos_nrl[k], state.pwr_nrl[k]
```

`rays\_end[i]` は実行時 `nraymax` 本数にかかわらず常に長さ 9 (`NEQ+1`) です。

## FAQ

### ■ [FAQ] Q. レイが全く動かない

`ANGTIN` / `ANGPHIN` / `RFIN` の初期条件が適切か確認してください。  
特に `ANGPHIN=0` だと軸対称でレイが磁軸に沿ってしまうことがあります。

### ■ [FAQ] Q. `rays\_end[i]

` の 9 要素は何?

`RAYS(0:NEQ, i)` に対応し, 0:位置, 1:波数, 2..:状態変数 等の

順です。詳細は `wr/wrcomm.f90` の定義をご覧ください。

## 変更履歴

[表]

| lllp{6.5cm}@{}}

Phase | PR | 日付 | 内容

L-0 | #-- | 2026-04-18 | `wr\_iter\_lhcd`,  
`wr\_test001`, `wr\_tst2\_ec` ベースライン

L-1 | #-- | 2026-04-18 | Makefile 分割

L-2 | #-- | 2026-04-18 | C ABI スタブ + `wr`  
\_api.h`

L-3 | #-- | 2026-04-18 | パラメータレジストリ

L-4 | #-- | 2026-04-18 | `libwrapi.so`

L-5 | #-- | 2026-04-18 | Python ラッパ `pyt`  
hon/wrlib/`

L-6 | #-- | 2026-04-18 | 4 層テスト `wrlib\_\*`  
、

L-7 | #-- | 2026-04-18 | ドキュメント |

[/表]

## テスト

```
bash test_run/run_tests.sh wrlib_equivalence wrlib_c_abi \
wrlib_ffi wrlib_wrapper wrlib_sweep
```

## 第 wrx モジュール 章

### ■ [学習目標]

本章では, TASK/WRX (拡張レイトレーシング, 粒子種ごとのパワー分解が可能) を Python から呼ぶ `wrxlib` の使い方を学びます.  
`wrxlib` と姉妹パッケージで, 同時に import できます.

### 概要 — wrx は何をする

{TASK/WRX} は WR を拡張し, 吸収パワーを{粒子種ごと}に分解したり, より詳細な O/X モード変換を扱ったりします. 出力は WR と似ていますが, `pwr\_nsa` (粒子種ごとのパワー), `pwr\_nsa\_nray[i][j]` (レイ i, 種 j のパワー) という 2 次元量が加わります.

### 前提とビルド

```
$ make -C lib libs_pic
$ make -C pl libs_pic
$ make -C eq libs_pic
$ make -C dp libs_pic
$ make -C mtxp libs_pic
$ make -C bpsd libs_pic
$ make -C wrx libwrxapi.so
```

### 最短 hello-world

```
from wrxlib import Wrxlib
with Wrxlib() as wrx:
 wrx.set_params(MODELG=2, RR=6.2, BB=5.3, NSMAX=2, NRAYMAX=1)
 wrx.set_param("RFIN[1]", 170.0e3) # 170 GHz
 wrx.run(nstpmax=0) # wr_run と同じく 0 で既定
 state = wrx.get_state()
 print("pwr_tot =", state.scalars["pwr_tot"])
 print("per-species pwrmax_rs =", state.pwrmax_rs_nsa)
```

## 既知の制約 --- `wrx\_run`

と libgrf`

### ■ [注意]

L-4 時点の `libwrxapi.so` は, `wrcalpwrf90` から  
`libgrf::grd1d` への{動的依存}が残っています.  
共有ライブラリ経由で `wrx\_run` を呼ぶと  
セグメンテーション違反を起こす可能性があります.

回避策:

- テストは `WRX\_RUN\_OK=1` 環境変数でのみ有効化  
(`TestWrxlibRun`).
- フルの `wrx\_run` 相当は Layer 1 (`wrx/wrxregress`)  
か静的リンクの Layer 2 C ハーネスを使う.

## `WrxState`

の構造`

```
state.nraymax, state.nstpmax, state.nsamax, state.nsmax
state.modelg, state.mdlwrq
state.scalars["pwr_tot"]
state.nstp_end[i] # レイ終了ステップ (別名 nstpmax_
 nray)
state.pwr_nray[i] # レイごとの総吸収パワー
state.pwr_nsa[j] # 種ごとの総吸収パワー
state.pwr_nsa_nray[i][j] # [レイ][種] の 2D
state.pos_pwrmax_rs_nsa[j] # 種ごとのピーク位置 (rs)
state.pwrmax_rs_nsa[j] # そのパワー値
state.pos_pwrmax_rl_nsa[j], state.pwrmax_rl_nsa[j]
```

## FAQ

### ■ [FAQ] Q. `wrlib` と `wrxlib` は共存可能?

はい, C シンボル接頭辞 `wr\_` と `wrx\_` が違うので  
同じ Python プロセスで両方 import して使えます.

### ■ [FAQ] Q. run がセグフォる

上記「既知の制約」参照. Layer 1 driver (`wrx/wrxregress`)  
または静的リンクの C ハーネスを使ってください.

## 変更履歴

Phase L-0/L-1/L-2/L-3/L-4/L-5 はすべて {2026-04-18} に実施.  
L-6 (4 層テスト) と L-7 (ドキュメント) も 2026-04-18  
. PR 番号は  
develop ログの `feature/wrx-library-\*` 系列.

## テスト

```
bash test_run/run_tests.sh wrx_iter01 wrx_demo
python 側: wrxlib のユニットテストは tests/ 配下を unittest で
PYTHONPATH=python python3 -m unittest discover python/wrxlib
/tests -v
```



## 第 fp モジュール 章

### ■ [学習目標]

本章では TASK/FP (Fokker-Planck 解析) を Python から呼ぶ `fplib` の使い方を学びます. 他モジュールにはない便利機能として, `set\_params` が dict や list を受け付け, 配列パラメータを一気に与えられます.

### 概要 — fp は何をする

{TASK/FP} はプラズマ中の粒子の{速度分布関数}を解く Fokker-Planck ソルバです. 波動加熱, NBI, プレムシュトラールング, シンクロトロン輻射などによる速度空間の変化を追跡します. 出力は `[NSAMAX][NRMAX]` の 2 次元プロファイルで, 密度 (`RNT`), 蓄積エネルギー (`RWT`), 温度 (`RTT`), 電流 (`RJT`) などが得られます.

### 前提とビルド

```
$ cd /home/you/task
$ make -C fp libs_pic # 自動で下流の PIC を全部作る
$ make -C fp libfpapi.so
```

### 最短 hello-world

```
from fplib import Fplib
with Fplib() as fp:
 fp.set_params(
 MODELG=3, NSMAX=3,
 PA={2: 2.0, 3: 3.0}, # 配列要素を dict で一気に
 PN={1: 0.8, 2: 0.4, 3: 0.4},
 NRMAX=40, NPMAX=50, NTHMAX=50,
 NTMAX=2, DELT=1.0e-3,
 RMIN=0.4, RMAX=0.8,
)
```

```

fp.run(ntmax=2)
state = fp.get_state()
print(f"TIMEFP={state.timefp:.6e} NRMAX={state.nrmax} NSAM
AX={state.nsamax}")
print(f"RNT[NSA=1, NR=1..5] = {state.RNT[0][:5]}")

```

``set_params``

拡張

`fp` だけは ``set_params`` が次の 3 形式を受け付けます:

- スカラー: ``RR=3.0``
- dict: ``PA={2: 2.0, 3: 3.0}`` → ``PA[2]=2.0, PA[3]=3.0``
- list: ``PN=[0.8, 0.4, 0.4]`` → ``PN[1]=0.8, PN[2]=0.4, PN[3]=0.4``

## ``FpState``

の主なフィールド

```

state.nrmax, state.nsamax, state.npmax, state.nthmax, state.
ntg2
state.timefp # 現在時刻
state.RNT, state.RWT, state.RTT, state.RJT, state.RPCT, stat
e.RPWT
それぞれ [NSAMAX][NRMAX] の list of list

```

## FAQ

### ■ [FAQ] Q. ``fp_finalize`` で確保メモリが残る?

L-3 時点では ``fp_allocate``/``fp_deallocate`` が非対称で, ``fp_finalize`` でも FPCOMM 配列は解放されません

1 プロセスにつき ``fp_init`` → ``fp_run`` → ``fp_finalize`` を 1 サイクルだけ回す運用を推奨します.

### ■ [FAQ] Q. 大規模メッシュが欲しい

``FP_MAX_NRMAX=100``, ``FP_MAX_NSAMAX=8`` はヘッダ

`fp/fp\_api.h` の `#define` で、構造体サイズに直結するコンパイル時キャップです. 大きくしたい場合は書き換えて `libfpapi.so` を再ビルドしてください.

### ■ [FAQ] Q. 複数コアを使いたい

FP は MPI / OpenMP の対応は{しません} (Python API 経由では).  
Python の `multiprocessing` で{プロセス分離}してパラメータスキャンするのが推奨経路です.

## 入力パラメータ表 (抜粋)

[表]

| p{3cm}p{2.2cm}p{9.2cm}@{}  
キャプション: fp モジュールの主な登録パラメータ

{名前} | {型} | {意味}

`RR, RA, RB, RKAP, RDLT, BB, RIP` | double | 装置幾何

`NRMAX, NPMAX, NTHMAX, NTMAX, NAVMAX` | int | メッシュ

`NSMAX, NSAMAX, NSBMAX` | int | 粒子種カウント

`NS\_NSA[i]`, `NS\_NSB[i]` | int 1D | 種マッピング

`PA[i]`, `PZ[i]`, `PN[i]`, `PNS[i]`, `PTR[i]`, `PTPP[i]`, `PTS[i]`, `PMAX[i]` | double 1D | 粒子種ごと

`DELT, EPSFP, LMAXFP` | 数値 | 時間発展

`R1, DELR1, RMIN, RMAX, E0, ZEFF` | double | 半径メッシュ \ | スカラー物理量

`PABS\_EC, PABS\_LH, PABS\_FW, PABS\_WR, PABS\_WM, RF\_WM` | double | 波動加熱

`MODELG, MODELE, MODELR, MODELS, MODELD`  
| int | モデル選択

`MODEL\_NBI, MODEL\_WAVE, MODEL\_DISRUPT, M  
ODEL\_BS, MODEL\_LOSS, MODEL\_SYNCH, MODEL\_  
FOW` | int | 各種物理機構

`MODEL\_C[i], MODEL\_W[i]` | int 1D | 粒子種ごとの  
衝突/波動モデル |  
[/表]

## 変更履歴 — Phase L-0 ~ L-7

[表]  
| lllp{6.5cm}@{}

Phase | PR | 日付 | 内容

L-0 | #13 | 2026-04-18 | `fp\_iter01`, `f  
p\_jt60`, `fp\_dt1` ベースライン

L-1 | #26 | 2026-04-18 | `fp/Makefile` C  
ORE/GRAPHICS/MENU 分割

L-2 | #29 | 2026-04-18 | `fp\_api.h` + 5  
関数スタブ

L-3 | #37 | 2026-04-18 | パラメータレジストリ 40 名  
, 55 変数

L-4 | #43 | 2026-04-18 | `libfpapi.so` +  
`fp\_graphics\_stubs.f90`

L-5 | #55 | 2026-04-18 | Python `python/  
fplib/`: dict/list set\_params 拡張

L-6 | #56 | 2026-04-18 | 4 層テスト `fplib\_\*`  
,

L-7 | #68 | 2026-04-18 | ドキュメント一式 |  
[/表]

## テスト

```
bash test_run/run_tests.sh fplib_equivalence fplib_c_abi \
```

fplib\_ffi fplib\_wrapper fplib\_sweep

## 第 テストインフラ 章

### ■ [学習目標]

本章では「書いた変更が正しく動いているか」を確認する 4 層テスト基盤を説明する. 以降は だ・である調 で簡潔に記述する.

### 4 層構成の意図

Phase L-6 では, 各モジュールに対して次の{4 層}のテストを揃えた.

- {Layer 1 — Equivalence (数値等価性)}  
`libXapi.so` 経由で計算した結果が, Phase 0  
で保存した  
JSON ベースラインと `1e-10` 以内で一致するか.
- {Layer 2 — C ABI (シンボル面)}  
C から直接 `.so` を叩く回帰: smoke / para  
m / run /  
run\_so / negative.
- {Layer 3 — Python ラッパ}  
`\_ffi` の ctypes 型整合, 高レベル `Xlib`  
クラスの  
ライフサイクル, 例外マッピング.
- {Layer 4 — Sweep (煙テスト)}  
3×3 のパラメータ格子 (例: `RR` × `BB`) を  
回し, 落ちずに完走することを確認.

### `run\_tests.sh`

と test\_definitions.conf`

テストは `test\_run/run\_tests.sh` に集約されている.  
定義は `test\_run/test\_definitions.conf` の 1  
行 1 テスト  
フォーマット:

```
TEST_NAME:MODULE:INPUT:DEPENDS:TIMEOUT:DESCRIPTION
```

Phase L-6 で新設された `MODULE=python` は「第 3 列

を  
 `PYTHONPATH=python` 付きでシェル実行する」特別モードである。  
 これにより `make -C tr tr\_api\_check\_all` や  
 `python3 -m unittest trlib.tests.test\_tr`  
 `lib` 等を  
 既存のテスト基盤に{一様に}乗せられる。

```
trlib_c_abi:python:make -C tr tr_api_check_all:none:300:Layer 2
r 2 C ABI suite
trlib_wrapper:python:python3 -m unittest -v trlib.tests.test
_trlib:none:120:Layer 3
trlib_ffl:python:python3 -m unittest -v trlib.tests.test_ffl
:none:60:Layer 3 (low)
trlib_equivalence:python:python3 -m unittest -v trlib.tests.
test_equivalence:none:300:Layer 1
trlib_sweep:python:python3 -m unittest -v trlib.tests.test_s
weep:none:180:Layer 4
```

## Layer 1 — Equivalence テスト

Phase 0 (L-0) で収集した `test\_run/baselines/`  
 `X\_\*.json` を  
 ground truth として、ライブラリ版の出力と `compare\_me`  
 `trics.py`  
 で diff する。許容誤差 `1e-10`。

比較対象は `XState.to\_dict()` のフィールド全て。ベースライ  
 ン  
 側で pin しているケース (`tr\_m0904`, `ti\_ar`) は  
 `1e-8` レベルのドリフトを{そのまま}許容。

## Layer 2 — C ABI テスト

各モジュール下 `X/tests/c\_abi/` に次の C プログラムが並ぶ。

- `test\_smoke.c` — 5 関数呼び出し順序, 終了コード
- `test\_param.c` — `X\_set\_param` 正常系
- `test\_run.c` — 静的リンク時の `X\_run`
- `test\_run\_so.c` — `dlopen` → 関数ポインタ経由
- `test\_negative.c` — エラーコード (1 / 2) を意図的に誘発

`make -C X X\_api\_check\_all` で全て走る.

## Layer 3 — Python ラッパ

`python/Xlib/tests/` に `test\_ffl.py` と  
`test\_Xlib.py` がある. ctypes の `Structure`  
フィールド順,  
`load\_library` の候補探索, 例外マッピングを  
`unittest` で検証. `.so` が無ければ FFI-必須テストは  
スキップされる.

## Layer 4 — Sweep テスト

`test\_sweep.py` は  $3 \times 3$  のパラメータ格子を回して, 戻り値  
`(>0\`  
に落ちないことを見る「煙テスト」. 数値一致は要求しない.

## テストの走らせ方

```
1 つだけ
bash test_run/run_tests.sh trlib_equivalence

モジュール別にまとめて
bash test_run/run_tests.sh trlib_equivalence trlib_c_abi trlib_ffl \
 trlib_wrapper trlib_sweep

Phase 0 の古典テストも含めて
bash test_run/run_tests.sh tr_iter01 tr_tst2 tr_m0904 \
 trlib_equivalence trlib_wrapper
```



## 第 作業状況・残作業 章

### ■ [学習目標]

本章は「どこまで進んだか, 何が残っているか」の現況報告である.  
だ・である調.

### モジュール別完了マトリクス

[表]  
| lccccccc@{}  
Module | L-0 | L-1 | L-2 | L-3 | L-4 | L-5 | L-6 | L-7  
tr | OK | OK | OK | OK | OK | OK | OK | OK  
ti | OK | OK | OK | OK | \$TRI^{1}\$ | OK  
| OK | OK  
wr | OK | OK | OK | OK | OK | OK | OK | OK  
wrx | OK | OK | OK | OK | OK\$^{2}\$ | OK  
| \$TRI^{2}\$ | OK  
fp | OK | OK | OK | OK | OK | OK | OK | OK  
eq | OK | plan | plan | plan | plan | plan | plan  
an | plan | plan  
tot | -- | plan | plan | plan | plan | plan | plan  
lan | plan | plan  
[/表]

脚注:  
[\$^{1}\$] `ti/libtiapi.so` のトップレベル Makefile  
ターゲットはフィーチャブランチ待ち.

`[$^2$] `wrx_run` が `libgrf::grdld` 依存で  
 .so 経由だとセグフォる. `WRX_RUN_OK=1` でテストを限定  
 有効化.`

## eq モジュールが除外されている理由

{eq} (平衡解析) モジュールは, 2026-04-18 時点で  
 Phase 0 (L-0) のみ完了しており, L-1 以降は{計画のみ}  
 (`docs/superpowers/plans/2026-04-\*eq-library-L\*.md`).

主な未解決事項:

- eq は Fortran 77 の fixed-form が大部分を占め,  
 F90 化 (`docs/superpowers/plans/  
 ../eq-f90-modernization-plan.md`) が未着手.
- Shim 方針 (F77 → F90 の段階移行と, F90 shim の除却計画) が  
 `eq-f90-shim-policy.md` で合意済みだが,  
 実装未着手.
- COMMON ブロックのモジュール化, BPSD 連携見直しが前提.

これらは本マニュアルの改訂時に追補される.

## tot モジュールが除外されている理由

{tot} (統合シミュレータ) は eq/tr/ti/fp/wr を束ねるトップレベル  
 で, 下位モジュールすべての L-\* が揃ってから最後にライブラリ化する  
 設計方針である. 依存関係グラフが DAG として確定するのを待つ.

## Shim ポリシー

`X\_graphics\_stubs.f90` は, 共有ライブラリ版に{不要な  
 グラフィクス関数}の{空実装}を置き, リンク時の「undefined  
 reference」を防ぐためのファイルである. `tr2` (CLI バイナリ)  
 ) は `libtrgrf.a` を静的リンクするので stubs は使われない.

将来, グラフィクスを C ABI 経由で呼ぶ計画 (未着手) が入るときに

stubs は撤去される.

## F90 モダナイゼーション

`trcom0.inc` / `trcom1.inc` といった F77 の `include` ファイルはモジュール化 (`trcomm.f90` とその派生) 済み. `eq` は未実施. `tot` は依存が落ち着いてから.

## その他の残作業

- 文字列パラメータ: `KNAMEQ` 以外は未対応. `wrx` の `KID\_NS`, `fp` の `KNAMFP` など.
- `fp\_finalize` の配列解放 (非対称 `allocate/deallocate`).
- メッシュキャップ (`FP\_MAX\_NRMAL=100` など) の拡大.
- マルチインスタンス化 (現状は 1 プロセス 1 インスタンス).
- `wrx\_run` の `libgrf` 依存解消.
- `ti/libtiapi.so` の develop 昇格.

## 第 付録 章

### Makefile ターゲット一覧

[表]

| p{6cm}p{8cm}@{}

キャプション: ライブラリ化に関連する主な Makefile ターゲット

{ターゲット} | {役割}

`make -C lib libs\_pic` | `lib/` の PIC .a

`make -C pl libs\_pic` | `plcomm\_pic.a`  
など

`make -C eq libs\_pic` | `libeq\_pic.a`

`make -C ob libs\_pic` | `libob\_pic.a` (  
fp/wr 経路)

`make -C dp libs\_pic` | `libdp\_pic.a` (  
wr/wrx/fp)

`make -C mt xp libs\_pic` | `libmt xp\_pic.a`

`make -C bpsd libs\_pic` | `libbpsd\_pic.a`

`make -C adpost libs\_pic` | `lib-adpost\_  
pic.a` (ti)

`make -C open-adas/adf11/adf11-lib libs\_  
pic` | adf11 PIC (ti)

`make -C tr libtrapi.so` | tr 共有ライブラリ

`make -C ti libtiapi.so` | ti 共有ライブラリ (  
FB 待ち)

`make -C wr libwrapi.so` | wr 共有ライブラリ

`make -C wrx libwrxapi.so` | wrx 共有ライブラリ

`make -C fp libfpapi.so` | fp 共有ライブラリ

```
`make -C X X_api_check_all` | Layer 2 C
ABI 回帰 (各 X)
```

```
`bash test_run/run_tests.sh` | 全テスト (Phase 0 + 4 層) |
[/表]
```

## 主な PR 一覧

```
tr: #2, #21, #27, #33, #35, #44, #53. ti
: #12, #22, #28, #34,
#41, #47, #57, #64. fp: #13, #26, #29, #
37, #43, #55, #56, #68.
wr/wrx の PR 番号は `git log --oneline devel
op --grep="Phase L-"
`で確認.
```

## 参考資料

- 設計仕様: `docs/superpowers/specs/2026-04-17-tr-library-design.md`
- アーキテクチャ図: `docs/{tr,ti,fp}\-library/architecture.md`
- フェーズ計画: `docs/superpowers/plans/2026-04-18-{tr,ti,wr,wrx,fp}\-library-L\*.md`
- CHANGELOG: リポジトリ root の `CHANGELOG.md`
- トップ README: `README`, `INSTALL\_Ubuntu.md`

## 用語集

[ABI] Application Binary Interface. バイナリレベルの関数呼び出し規約.

[C ABI] C 言語の呼び出し規約. Fortran `BIND(C)` や Python `ctypes` で橋渡しの基準になる.

[ctypes] Python 標準ライブラリ. `.so` / `.dll` を読み込み C 関数を呼べる.

[dataclass] Python の `@dataclass`. フィールド

付きの値クラスを

定義するデコレータ.

[dlopen / RTLD\_LAZY] 実行時にライブラリを読み込む POSIX API.

LAZY は「シンボル解決を初回呼び出しまで遅延」.

[COMMON ブロック] Fortran の古典的なグローバル変数機構. TR /FP 等は

内部で多用しているため, 1 プロセス 1 インスタンス制限の原因になる.

[PIC] Position Independent Code. `so` に  
入れるには必要.

[namelist] Fortran のテキスト入力フォーマット (`&TR .  
.. /`).

[Phase L] 本プロジェクトでの「ライブラリ化」相を指す L-0..L-7  
の  
8 段階.

## 連絡先・貢献

- 主担当: k-yoshimi (東京大学)
- email: k-yoshimi@g.ecc.u-tokyo.ac.jp
- リポジトリ: private fork of TASK (京大 BPSI).

## 第 補足: Python 最小例カタログ 章

本付録では, 5 モジュールそれぞれの{最短 Python コード}を再掲する. コピペして `python3 script.py` すればそのまま動くことを目指した.

### tr

```
from trlib import Trlib
with Trlib() as tr:
 tr.set_params(RR=3.0, BB=3.0, NSMAX=2, DT=0.05, NTMAX=50)
 tr.set_param("PN[1]", 1.0)
 tr.set_param("PT[1]", 1.5)
 tr.run(ntmax=50)
 s = tr.get_state()
 print("NT", s.nt, "WPT", s.scalars["WPT"])
```

### ti

```
from tilib import TiLib
with TiLib() as ti:
 ti.set_params(RR=3.0, BB=3.0, NSMAX=2, DT=0.01, NRMAX=10, NTMAX=5)
 ti.run(ntmax=5)
 s = ti.get_state()
 print("T", s.T, "RTA[0]", s.RTA[0])
```

### Wr

```
from wrlib import Wrlib
with Wrlib() as wr:
 wr.set_params(RR=3.0, BB=3.5, NSMAX=2)
 wr.run(nray_request=0)
 s = wr.get_state()
 print("NRAYMAX", s.nraymax, "pwrmax_rs", s.scalars["pwrmax_rs"])
```

### WRX

```
from wrxlib import Wrxlib
with Wrxlib() as wrx:
 wrx.set_params(MODELG=2, RR=6.2, BB=5.3, NSMAX=2, NRAYMA
X=1)
 wrx.set_param("RFIN[1]", 170.0e3)
 # wrx.run(nstpmax=0) # libgrf 依存のため環境で有効化
 s = wrx.get_state()
 print("nsamax", s.nsamax, "pwr_tot", s.scalars["pwr_tot"])
```

## fp

```
from fplib import Fplib
with Fplib() as fp:
 fp.set_params(
 MODELG=3, NSMAX=3,
 PA={2: 2.0, 3: 3.0}, PN={1: 0.8, 2: 0.4, 3: 0.4},
 NRMAX=40, NPMAX=50, NTHMAX=50, NTMAX=2, DELT=1.0e-3,
 RMIN=0.4, RMAX=0.8,
)
 fp.run(ntmax=2)
 s = fp.get_state()
 print("timefp", s.timefp, "RNT[0][:3]", s.RNT[0][:3])
```



## 第 補足: C から直接呼ぶ例 章

Python を介さず, C プログラムから `.so` を dlopen する例.

```
#include <stdio.h>
#include <dlfcn.h>
#include "tr_api.h"

int main(void) {
 void* lib = dlopen("tr/libtrapi.so", RTLD_LAZY);
 int (*tr_init)(void) = dlsym(lib, "tr_init");
 int (*tr_run)(int) = dlsym(lib, "tr_run");
 int (*tr_set_param)(const char*, double) = dlsym(lib, "tr_set_param");
 int (*tr_get_state)(tr_state_t*) = dlsym(lib, "tr_get_state");
 int (*tr_finalize)(void) = dlsym(lib, "tr_finalize");

 if (tr_init() != 0) { fprintf(stderr, "tr_init failed");
return 1; }
 tr_set_param("RR", 3.0);
 tr_set_param("BB", 3.0);
 if (tr_run(10) != 0) { fprintf(stderr, "tr_run failed");
return 1; }
 tr_state_t s;
 tr_get_state(&s);
 printf("NT=
tr_finalize();
dlclose(lib);
return 0;
}
```

コンパイル:

```
gcc -I tr -o mydriver mydriver.c -ldl
./mydriver
```

## 第 補足: 回帰テストのテンプレート 章

---

Phase 0 で保存されるベースライン JSON と, それに対する  $1e-10$  比較のテンプレート.

```
import json
from trlib import Trlib

1. 基準データ読み込み
with open("test_run/baselines/tr_iter01.json") as f:
 base = json.load(f)

2. ライブラリ経由で同じシナリオを回す
with Trlib() as tr:
 for k, v in base["params"].items():
 if "[" in k:
 tr.set_param(k, v)
 else:
 tr.set_params(**{k: v})
 tr.run(ntmax=base["params"]["NTMAX"])
 out = tr.get_state().to_dict()

3. スカラー diff
tol = 1e-10
for key, ref in base["scalars"].items():
 cur = out["scalars"][key]
 assert abs(cur - ref) <= tol * max(1.0, abs(ref)), (key,
cur, ref)
print("equivalence OK")
```

## 第 補足: TikZ 依存関係グラフ 章

---

L-\* フェーズ間の依存関係 (各モジュール共通):

[図 (TikZ): PDF 版では LaTeX 層で描画]

## 第 編集後記 章

---

toc{chapter}{編集後記}

本書は 2026 年 4 月 18 日の集中開発(Phase L-0 ～ L-7)の成果を,  
{初学者が一人で読み進められるように}整えたマニュアルである.  
共有ライブラリや ctypes の話は一般に難解な分野だが, 「なぜそうするか」  
を丁寧に説明することで, 物理屋の読者が{自分のスクリプト}を  
数分で書き始められる状態を目指した.

ご意見・誤記の指摘は k-yoshimi@g.ecc.u-tokyo.ac.jp  
まで.

TASK プラズマ輸送ライブラリ化プロジェクト --- 日本語マニュアル

(tr / ti / wr / wrx / fp, Phase L-0 ～ L-7)

© 2026 k-yoshimi. TASK コードの license に準じる

.